WebSocket – Complete Notes (Beginner to Advanced)
English + বাংলা | Python | Django | FastAPI | Redis

Table of Contents

1. What is WebSocket?
2. Why Do We Need WebSocket?
3. HTTP vs WebSocket
4. WebSocket Protocol
5. How WebSocket Works
6. WebSocket Handshake
7. WebSocket Architecture
8. Important WebSocket Terminologies
9. WebSocket Lifecycle
10. JavaScript WebSocket API
11. WebSocket with Python
12. WebSocket with FastAPI
13. WebSocket with Django Channels
14. WebSocket Groups and Broadcasting
15. Redis with WebSocket
16. WebSocket Authentication
17. WebSocket Security
18. Polling vs Long Polling vs SSE vs WebSocket
19. Ping and Pong
20. Error Handling and Reconnection
21. WebSocket Performance Optimization
22. WebSocket Scaling
23. WebSocket Use Cases
24. WebSocket Best Practices
25. Common WebSocket Errors
26. WebSocket Interview Questions
27. WebSocket Project Ideas
28. WebSocket Learning Roadmap
29. Important Commands
30. Final Summary

---

1. What is WebSocket?

Definition

WebSocket is a communication protocol that provides full-duplex, real-time communication between a client and a server over a single persistent connection.

বাংলায়:

WebSocket হলো একটি communication protocol, যার মাধ্যমে Client এবং Server-এর মধ্যে একটি স্থায়ী connection তৈরি হয়।

এই connection ব্যবহার করে Client এবং Server দুজনেই যেকোনো সময় একে অপরকে message পাঠাতে পারে।

Example

ধরো, তুমি একটি Chat Application তৈরি করেছ।

User A একটি message পাঠাল:

"Hello"

WebSocket ব্যবহার করলে Server সঙ্গে সঙ্গে User B-কে message পাঠিয়ে দিতে পারবে।

এখানে User B-কে বারবার Server-কে request করতে হবে না।

Main Features

- Full-duplex communication
- Persistent connection
- Real-time data transfer
- Low communication overhead
- Supports text and binary messages
- Suitable for real-time applications

Full-Duplex Communication

Full-duplex means both sides can send and receive data simultaneously.

বাংলায়:

Client এবং Server একই সময়ে data পাঠাতে ও গ্রহণ করতে পারে।

Example

```text
Client  <----------------->  Server

Send Message  ------------>
```

```
        <------------  Receive Message

Send Message  ------------>
        <------------  Receive Message
```

---

2. Why Do We Need WebSocket?

HTTP সাধারণত Request-Response model অনুসরণ করে।

Client request পাঠায়, Server response দেয়।

কিন্তু কিছু application-এ Server থেকে Client-কে তাৎক্ষণিক data পাঠানো প্রয়োজন।

যেমন:

- Chat applications
- Live notifications
- Online gaming
- Live stock prices
- Real-time dashboards
- Collaborative editors
- Live tracking systems

HTTP Problem

ধরো, তুমি একটি chat application তৈরি করেছ।

User B নতুন message পেয়েছে কি না জানতে User A-এর browser-কে বারবার request পাঠাতে হবে।

এটিকে Polling বলা হয়।

```text
Client ---- Request ----> Server
Client <--- Response ---- Server

Client ---- Request ----> Server
Client <--- Response ---- Server

Client ---- Request ----> Server
Client <--- Response ---- Server
```

এতে অনেক unnecessary request তৈরি হয়।

WebSocket Solution

WebSocket connection একবার তৈরি হলে সেটি খোলা থাকে।

Server প্রয়োজন হলেই Client-কে message পাঠাতে পারে।

```text
Client <================> Server

    Persistent Connection

    Message
    Notification
    Live Update
```

Benefits

1. কম communication overhead
2. Real-time updates
3. Persistent connection
4. Bidirectional communication
5. Efficient message delivery

---

3. HTTP vs WebSocket

Feature	HTTP	WebSocket
Communication	Request-Response	Full-Duplex
Connection	সাধারণত request ভিত্তিক	Persistent
Direction	Client initiates requests	Both sides can send
Real-time	Polling প্রয়োজন হতে পারে	Native real-time communication
Overhead	প্রতিটি request-এ HTTP overhead	Connection setup-এর পর কম overhead
Protocol	HTTP	WebSocket
Use Case	REST API	Chat, Live Notification

HTTP Example

```http
```

```
GET /api/users HTTP/1.1
Host: example.com
```

Server response:

```http
HTTP/1.1 200 OK
Content-Type: application/json

{
   "name": "Faruk"
}
```

WebSocket Example

```text
Client: Hello
Server: Welcome
Client: How are you?
Server: I am fine.
```

এখানে একই connection ব্যবহার করে দুই পক্ষই message পাঠাতে পারে।

---

4. WebSocket Protocol

WebSocket-এর official protocol identifier:

```text
ws://
wss://
```

ws://

Unencrypted WebSocket connection।

Example:

```text
```

ws://localhost:8000/ws/chat/
```

## wss://

Encrypted WebSocket connection।

এটি TLS ব্যবহার করে secure communication প্রদান করে।

Example:

```text
wss://example.com/ws/chat/
```

Production application-এ সাধারণত `wss://` ব্যবহার করা উচিত।

WebSocket Port

WebSocket নিজস্ব কোনো নির্দিষ্ট port বাধ্যতামূলক করে না।

সাধারণত:

- HTTP: 80
- HTTPS: 443

WebSocket একই port-এ HTTP Upgrade handshake ব্যবহার করতে পারে।

---

5. How WebSocket Works

WebSocket communication সাধারণত নিচের ধাপগুলো অনুসরণ করে:

Step 1: Client Connection Request

Client Server-এর কাছে WebSocket connection request পাঠায়।

Step 2: Handshake

Client এবং Server-এর মধ্যে HTTP-based handshake হয়।

Step 3: Connection Established
```

Server handshake গ্রহণ করলে connection তৈরি হয়।

Step 4: Data Exchange

Client এবং Server message আদান-প্রদান করে।

Step 5: Connection Closed

যেকোনো পক্ষ connection বন্ধ করতে পারে।

Diagram

```text
      CLIENT                SERVER
        |                     |
        |---- Handshake ------->|
        |                     |
        |<--- HTTP 101 ---------|
        |                     |
        |                     |
        |==== Connection =======|
        |                     |
        |---- Message --------->|
        |<--- Message ----------|
        |                     |
        |---- Message --------->|
        |<--- Message ----------|
        |                     |
        |---- Close ----------->|
        |                     |
        |<--- Close ------------|
        |                     |
```

---

6. WebSocket Handshake

WebSocket connection তৈরি করার জন্য প্রথমে HTTP Upgrade handshake ব্যবহার করা হয়।

Client Request

```http
GET /chat HTTP/1.1
```

Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: random-generated-key
Sec-WebSocket-Version: 13
```

### Important Headers

| Header | Description |
|---|---|
| Upgrade | Protocol upgrade request |
| Connection | Connection upgrade নির্দেশ করে |
| Sec-WebSocket-Key | Handshake security key |
| Sec-WebSocket-Version | WebSocket protocol version |

## Server Response

```http
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: generated-value
```

What is 101 Switching Protocols?

HTTP status code `101` বোঝায় Server protocol switching request গ্রহণ করেছে।

অর্থাৎ HTTP connection WebSocket communication-এ upgrade হয়েছে।

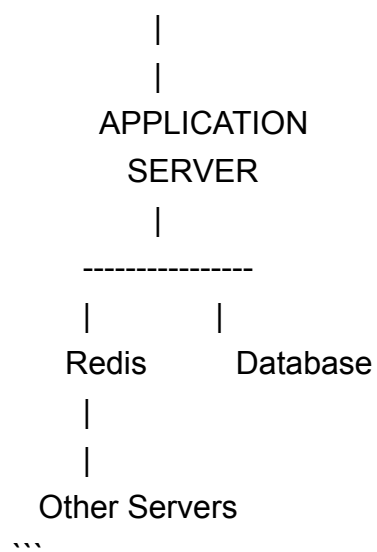
Note: এই HTTP/1.1 Upgrade পদ্ধতির পাশাপাশি HTTP/2-তে WebSocket connection স্থাপনের জন্য Extended CONNECT mechanism (RFC 8441) রয়েছে।

---

7. WebSocket Architecture

একটি সাধারণ WebSocket architecture:

```text
        CLIENT
          |
          |
```
```



Components

1. Client

যে application WebSocket connection তৈরি করে।

Example:

- Browser
- Mobile application
- Desktop application

2. WebSocket Server

Client connection পরিচালনা করে।

Example:

- FastAPI
- Django Channels
- Node.js WebSocket Server

3. Redis

একাধিক server-এর মধ্যে message exchange করতে সাহায্য করতে পারে।

4. Database

Permanent data সংরক্ষণ করে।

Example:

- PostgreSQL
- MySQL
- MongoDB

---

8. Important WebSocket Terminologies

1. Client

যে application Server-এর সঙ্গে connection তৈরি করে।

2. Server

যে application WebSocket connection গ্রহণ ও পরিচালনা করে।

3. Connection

Client এবং Server-এর মধ্যে communication channel।

4. Handshake

WebSocket connection তৈরি করার প্রাথমিক negotiation process।

5. Frame

WebSocket protocol-এর data transmission unit।

6. Message

এক বা একাধিক WebSocket frame দিয়ে তৈরি logical data unit।

7. Ping

Connection সচল আছে কি না পরীক্ষা করার control frame।

8. Pong

Ping-এর response হিসেবে পাঠানো control frame।

9. Close Frame

Connection বন্ধ করার জন্য ব্যবহৃত control frame।

10. Broadcasting

একটি message একাধিক connected client-এর কাছে পাঠানো।

11. Room / Group

একাধিক client-কে একটি logical group-এ রাখা।

12. Heartbeat

Connection সচল আছে কি না নিয়মিত পরীক্ষা করার পদ্ধতি।

---

9. WebSocket Lifecycle

WebSocket connection-এর সাধারণ lifecycle:

```text
CONNECTING
    |
    v
  OPEN
    |
    v
 CLOSING
    |
    v
  CLOSED
```

JavaScript-এ চারটি readyState রয়েছে।

State	Value	Meaning
CONNECTING	0	Connection তৈরি হচ্ছে
OPEN	1	Connection সক্রিয়
CLOSING	2	Connection বন্ধ হচ্ছে
CLOSED	3	Connection বন্ধ

Example:

```javascript
const socket = new WebSocket(
    "ws://localhost:8000/ws/"
);

console.log(socket.readyState);
```

---

10. JavaScript WebSocket API

Browser-এর built-in WebSocket API ব্যবহার করে WebSocket connection তৈরি করা যায়।

10.1 Create Connection

```javascript
const socket = new WebSocket(
    "ws://localhost:8000/ws/chat/"
);
```

10.2 onopen

Connection তৈরি হলে execute হয়।

```javascript
socket.onopen = function(event) {
    console.log("Connected!");
};
```

10.3 onmessage

Server থেকে message এলে execute হয়।

```javascript
socket.onmessage = function(event) {
    console.log("Message:", event.data);
};
```

10.4 onerror

Error হলে execute হয়।

```javascript
socket.onerror = function(error) {
    console.log("WebSocket Error:", error);
};
```

10.5 onclose

Connection বন্ধ হলে execute হয়।

```javascript
socket.onclose = function(event) {
    console.log("Connection Closed");
    console.log(event.code);
    console.log(event.reason);
};
```

10.6 send()

Server-এর কাছে message পাঠায়।

```javascript
socket.send("Hello Server");
```

JSON পাঠানো:

```javascript
const data = {
    username: "Faruk",
    message: "Hello"
};

socket.send(JSON.stringify(data));
```

10.7 close()

Connection বন্ধ করে।

```javascript
```

```
socket.close();
```

Complete JavaScript Example

```javascript
const socket = new WebSocket(
    "ws://localhost:8000/ws/chat/"
);

socket.onopen = () => {
    console.log("WebSocket Connected");
};

socket.onmessage = (event) => {
    console.log("Received:", event.data);
};

socket.onerror = (error) => {
    console.log("Error:", error);
};

socket.onclose = () => {
    console.log("WebSocket Disconnected");
};

function sendMessage(message) {
    if (socket.readyState === WebSocket.OPEN) {
        socket.send(message);
    } else {
        console.log("Connection is not open");
    }
}
```

---

11. WebSocket with Python

Python-এ WebSocket ব্যবহার করার জন্য `websockets` package ব্যবহার করা যায়।

Installation

```bash
```

```
pip install websockets
```

Simple WebSocket Server

```python
import asyncio
import websockets

async def handler(websocket):

    async for message in websocket:

        print("Received:", message)

        await websocket.send(
            f"Server received: {message}"
        )

async def main():

    async with websockets.serve(
        handler,
        "localhost",
        8765
    ):

        print("WebSocket server running")

        await asyncio.Future()

asyncio.run(main())
```

Python WebSocket Client

```python
import asyncio
import websockets

async def main():

    async with websockets.connect(
        "ws://localhost:8765"
```

```
    ) as websocket:

        await websocket.send("Hello Server")

        response = await websocket.recv()

        print(response)

asyncio.run(main())
```

Explanation

- `websockets.serve()` Server তৈরি করে।
- `websockets.connect()` Client connection তৈরি করে।
- `send()` message পাঠায়।
- `recv()` message গ্রহণ করে।
- `async for` incoming message পড়তে সাহায্য করে।

Note: `websockets` package-এর API version অনুযায়ী কিছু function signature আলাদা হতে পারে। উপরের উদাহরণটি আধুনিক API style অনুসরণ করে।

---

12. WebSocket with FastAPI

FastAPI-তে built-in WebSocket support রয়েছে।

Installation

```bash
pip install fastapi uvicorn
```

Project Structure

```text
project/
│
├── main.py
│
└── requirements.txt
```

Basic WebSocket Example

main.py

```python
from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(
    websocket: WebSocket
):

    await websocket.accept()

    while True:

        data = await websocket.receive_text()

        print("Received:", data)

        await websocket.send_text(
            f"Message received: {data}"
        )
```

Run Server

```bash
uvicorn main:app --reload
```

Browser Client

```html
<!DOCTYPE html>
<html>

<body>

    <input
        type="text"
        id="message"
```

```
    placeholder="Enter message"
  >

  <button onclick="sendMessage()">
    Send
  </button>

  <script>

    const socket = new WebSocket(
      "ws://localhost:8000/ws"
    );

    socket.onopen = () => {
      console.log("Connected");
    };

    socket.onmessage = (event) => {
      console.log("Server:", event.data);
    };

    function sendMessage() {

      const input = document.getElementById(
        "message"
      );

      socket.send(input.value);

      input.value = "";
    }

  </script>

</body>
</html>
```

FastAPI Important Methods

Method	Description
`accept()`	Connection গ্রহণ করে
`receive_text()`	Text message গ্রহণ করে

`receive_bytes()`	Binary data গ্রহণ করে
`receive_json()`	JSON data গ্রহণ করে
`send_text()`	Text পাঠায়
`send_bytes()`	Binary data পাঠায়
`send_json()`	JSON পাঠায়
`close()`	Connection বন্ধ করে

Error Handling

```python
from fastapi import (
    FastAPI,
    WebSocket,
    WebSocketDisconnect
)

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(
    websocket: WebSocket
):

    await websocket.accept()

    try:

        while True:

            message = await websocket.receive_text()

            await websocket.send_text(
                f"Received: {message}"
            )

    except WebSocketDisconnect:

        print("Client disconnected")
```

`WebSocketDisconnect` ব্যবহার করা হয় Client connection বিচ্ছিন্ন হলে সেটি handle করার জন্য।

---

13. WebSocket with Django Channels

Django-তে real-time WebSocket application তৈরি করতে Django Channels ব্যবহার করা হয়।

What is Django Channels?

Django Channels হলো Django-এর asynchronous communication support বাড়ানোর একটি extension।

এটি WebSocket, background communication এবং অন্যান্য asynchronous protocol পরিচালনা করতে সাহায্য করে।

Installation

```bash
pip install channels
```

Redis channel layer ব্যবহার করতে:

```bash
pip install channels-redis
```

Project Structure

```text
myproject/
│
├── manage.py
│
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── asgi.py
│   └── urls.py
│
└── chat/
    ├── __init__.py
    ├── consumers.py
    └── routing.py
```

Step 1: settings.py

```python
```

```
INSTALLED_APPS = [
    "channels",
    "chat",
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
]

ASGI_APPLICATION = "config.asgi.application"
```

Step 2: asgi.py

```python
import os

from django.core.asgi import get_asgi_application

from channels.routing import (
    ProtocolTypeRouter,
    URLRouter
)

from channels.auth import AuthMiddlewareStack

from chat.routing import websocket_urlpatterns

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)

django_asgi_app = get_asgi_application()

application = ProtocolTypeRouter({

    "http": django_asgi_app,

    "websocket": AuthMiddlewareStack(
        URLRouter(
            websocket_urlpatterns
```

```
        )
    ),

})
```

Step 3: consumers.py

Consumer হলো Django Channels-এর এমন একটি component, যা connection এবং message পরিচালনা করে।

```python
import json

from channels.generic.websocket import (
    AsyncWebsocketConsumer
)

class ChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        await self.accept()

        await self.send(
            text_data=json.dumps({
                "message": "Connected successfully"
            })
        )

    async def disconnect(self, close_code):

        print("Disconnected:", close_code)

    async def receive(self, text_data):

        data = json.loads(text_data)

        message = data.get("message", "")

        await self.send(
            text_data=json.dumps({
                "message": message
            })
        )
```

```

## Step 4: routing.py

```python
from django.urls import path

from .consumers import ChatConsumer

websocket_urlpatterns = [

    path(
        "ws/chat/",
        ChatConsumer.as_asgi()
    ),

]
```

Step 5: Run Server

Development-এর জন্য Daphne install করতে পারো:

```bash
pip install daphne
```

```bash
daphne -p 8000 config.asgi:application
```

Important Consumer Methods

Method	Description
`connect()`	Client connect হলে execute হয়
`disconnect()`	Client disconnect হলে execute হয়
`receive()`	Message receive হলে execute হয়
`send()`	Message পাঠায়
`accept()`	Connection গ্রহণ করে
`close()`	Connection বন্ধ করে

Sync vs Async Consumer
```

Sync Consumer

```python
from channels.generic.websocket import (
    WebsocketConsumer
)

class MyConsumer(WebsocketConsumer):

    def connect(self):
        self.accept()

    def receive(self, text_data):
        self.send(text_data=text_data)

    def disconnect(self, close_code):
        pass
```

Async Consumer

```python
from channels.generic.websocket import (
    AsyncWebsocketConsumer
)

class MyConsumer(AsyncWebsocketConsumer):

    async def connect(self):
        await self.accept()

    async def receive(self, text_data):
        await self.send(text_data=text_data)

    async def disconnect(self, close_code):
        pass
```

Difference

Sync Consumer	Async Consumer
Synchronous	Asynchronous
Blocking operations হতে পারে	Non-blocking I/O support করে

| Normal Django sync code-এর জন্য সুবিধাজনক | Async operations-এর জন্য সুবিধাজনক |

---

14. WebSocket Groups and Broadcasting

What is Broadcasting?

Broadcasting হলো একটি message একাধিক connected client-এর কাছে পাঠানো।

Example:

একটি chat room-এ ১০ জন user connected আছে।

একজন user message পাঠালে বাকি user-রা সেই message পাবে।

Django Channels Group

একটি group-এর মধ্যে একাধিক WebSocket connection রাখা যায়।

Consumer Example

```python
import json

from channels.generic.websocket import (
    AsyncWebsocketConsumer
)

class ChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        self.room_group_name = "chat_room"

        await self.channel_layer.group_add(
            self.room_group_name,
            self.channel_name
        )

        await self.accept()

    async def disconnect(self, close_code):
```

```
        await self.channel_layer.group_discard(
            self.room_group_name,
            self.channel_name
        )

    async def receive(self, text_data):

        data = json.loads(text_data)

        message = data["message"]

        await self.channel_layer.group_send(

            self.room_group_name,

            {
                "type": "chat_message",
                "message": message
            }

        )

    async def chat_message(self, event):

        await self.send(
            text_data=json.dumps({
                "message": event["message"]
            })
        )
```

Important Methods

group_add()

Client-কে group-এ যুক্ত করে।

```python
await self.channel_layer.group_add(
    group_name,
    channel_name
)
```

group_discard()

Client-কে group থেকে সরিয়ে দেয়।

```python
await self.channel_layer.group_discard(
    group_name,
    channel_name
)
```

group_send()

Group-এর সব member-এর কাছে event পাঠায়।

```python
await self.channel_layer.group_send(
    group_name,
    {
        "type": "chat_message",
        "message": "Hello"
    }
)
```

Channel Layer Event Type

```python
{
    "type": "chat_message"
}
```

এখানে `type` হলো handler method-এর নাম নির্ধারণ করে।

`chat_message` type-এর জন্য `chat_message()` method execute হবে।

---

15. Redis with WebSocket

What is Redis?

Redis হলো একটি in-memory data store, যা caching, messaging এবং অন্যান্য দ্রুত data operations-এর জন্য ব্যবহার করা হয়।

Django Channels-এ Redis channel layer হিসেবে ব্যবহার করা যায়।

Why Redis?

ধরো, তোমার application-এ দুটি WebSocket server রয়েছে।

```text
        CLIENTS
          |
     ------------
     |          |
   Server 1    Server 2
     |          |
     ------------
          |
        Redis
```

একটি Client Server 1-এর সঙ্গে connected।

অন্য Client Server 2-এর সঙ্গে connected।

Server 1 থেকে পাঠানো message Server 2-এর Client-দের কাছে পৌঁছাতে Redis channel layer সাহায্য করতে পারে।

Install Redis Channel Layer

```bash
pip install channels-redis
```

settings.py

```python
CHANNEL_LAYERS = {

   "default": {

       "BACKEND": (
          "channels_redis.core.RedisChannelLayer"
       ),
```

```
        "CONFIG": {

            "hosts": [
                "redis://127.0.0.1:6379/0"
            ],

        },

    },

}
```

Redis Responsibilities

- Channel layer messaging
- Cross-server communication
- Group broadcasting
- Temporary message distribution

Important Note

Redis channel layer permanent message storage-এর বিকল্প নয়।

Chat history, user information বা order information স্থায়ীভাবে রাখতে PostgreSQL বা অন্য database ব্যবহার করতে হবে।

---

16. WebSocket Authentication

WebSocket connection-এর ক্ষেত্রেও authentication প্রয়োজন।

কারণ যেকোনো user যেন অন্য user-এর private message বা protected data access করতে না পারে।

Authentication Methods

1. Session Authentication
2. Token Authentication
3. JWT Authentication
4. Custom Authentication Middleware

Django Channels Session Authentication

`AuthMiddlewareStack` ব্যবহার করলে WebSocket scope-এ session-based authenticated user পাওয়া যায়।

Example:

```python
class ChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        user = self.scope["user"]

        if user.is_authenticated:

            await self.accept()

        else:

            await self.close()
```

JWT Authentication

JWT ব্যবহার করলে token validate করতে হবে।

Concept:

```text
Client
  |
  | JWT Token
  |
WebSocket Server
  |
  | Validate Token
  |
Authenticated User
```

Important: Browser-এর native WebSocket constructor arbitrary custom headers সেট করার সুযোগ দেয় না।

তাই browser client-এর ক্ষেত্রে secure session cookie, অথবা carefully designed token authentication flow ব্যবহার করা যায়।

URL query string-এ token পাঠালে server, proxy বা access log-এ token থেকে যেতে পারে। তাই এটি এড়িয়ে চলা বা অত্যন্ত সতর্কভাবে পরিচালনা করা উচিত।

---

17. WebSocket Security

WebSocket application-এ security খুব গুরুত্বপূর্ণ।

1. Use WSS

Production-এ encrypted WebSocket ব্যবহার করো।

```text
wss://example.com/ws/
```

2. Authentication

প্রতিটি connection-এর user identity যাচাই করো।

3. Authorization

User যে room বা resource access করার অনুমতি রাখে শুধু সেটিই access করতে দাও।

4. Origin Validation

অপরিচিত website থেকে WebSocket connection গ্রহণ করা বন্ধ করো।

Django Channels-এ:

```python
from channels.security.websocket import (
    AllowedHostsOriginValidator
)

application = ProtocolTypeRouter({

    "http": django_asgi_app,

    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(websocket_urlpatterns)
        )
    ),
```

})
```

## 5. Input Validation

Client থেকে আসা message validate করো।

```python
message = data.get("message")

if not isinstance(message, str):
    return

if len(message) > 1000:
    return
```

6. Rate Limiting

একজন user যেন অতিরিক্ত message পাঠিয়ে server overload করতে না পারে।

7. Message Size Limit

অতিরিক্ত বড় message reject করো।

8. Avoid Sensitive Logging

Password, JWT token বা private message log করা উচিত নয়।

---

18. Polling vs Long Polling vs SSE vs WebSocket

1. Polling

Client নির্দিষ্ট সময় পরপর Server-কে request পাঠায়।

```text
Client ---> Server
Client ---> Server
Client ---> Server
```

Example:
```

```javascript
setInterval(async () => {

    const response = await fetch("/api/updates/");

    const data = await response.json();

    console.log(data);

}, 5000);
```

2. Long Polling

Client request পাঠায়।

Server data পাওয়া পর্যন্ত বা timeout হওয়া পর্যন্ত request ধরে রাখে।

Response পাওয়ার পর Client আবার request পাঠায়।

3. SSE (Server-Sent Events)

Server থেকে Client-এর দিকে একমুখী real-time event stream পাঠানো হয়।

Client সাধারণ HTTP connection ব্যবহার করে event গ্রহণ করে।

4. WebSocket

Client এবং Server উভয় দিক থেকে message পাঠানো যায়।

Comparison

Feature	Polling	Long Polling	SSE	WebSocket
Direction	Client request	Server response	Server to Client	Both directions
Persistent	No	Request-based	Yes	Yes
Real-time	Limited	Better	Yes	Yes
Browser support	Yes	Yes	Yes	Yes
Best for	Simple updates	Older real-time systems	Notifications	Chat and gaming

---

19. Ping and Pong

Ping এবং Pong WebSocket-এর control frame।

এগুলো connection সচল আছে কি না পরীক্ষা করতে সাহায্য করে।

Ping

একটি পক্ষ Ping পাঠায়।

Pong

অন্য পক্ষ Pong দিয়ে response দেয়।

```text
Client                Server

  |------ Ping --------->|
  |                      |
  |<----- Pong ----------|
  |                      |
```

Why Needed?

কখনো network disconnect হয়ে যায়, কিন্তু application বুঝতে পারে না।

Heartbeat mechanism ব্যবহার করলে dead connection শনাক্ত করা সহজ হয়।

Note: Browser-এর JavaScript WebSocket API সরাসরি Ping/Pong control frame তৈরি বা পড়ার interface দেয় না। সাধারণত browser implementation এগুলো protocol level-এ handle করে।

---

20. Error Handling and Reconnection

Network problem হলে WebSocket connection disconnect হতে পারে।

তাই reconnect logic থাকা গুরুত্বপূর্ণ।

Basic Reconnection Example

```javascript
let socket;
```

```
function connectWebSocket() {

    socket = new WebSocket(
        "ws://localhost:8000/ws/"
    );

    socket.onopen = () => {
        console.log("Connected");
    };

    socket.onmessage = (event) => {
        console.log(event.data);
    };

    socket.onclose = () => {

        console.log("Disconnected");

        setTimeout(
            connectWebSocket,
            3000
        );

    };

    socket.onerror = (error) => {
        console.log("Error:", error);
    };

}

connectWebSocket();
```

Exponential Backoff

প্রতিবার reconnect fail হলে অপেক্ষার সময় বাড়ানো হয়।

Example:

```text
Attempt 1: 1 second
Attempt 2: 2 seconds
Attempt 3: 4 seconds
```

Attempt 4: 8 seconds
Attempt 5: 16 seconds
```

## Improved Reconnection Example

```javascript
let socket;

let retryDelay = 1000;

const MAX_DELAY = 30000;

function connect() {

    socket = new WebSocket(
        "wss://example.com/ws/"
    );

    socket.onopen = () => {

        console.log("Connected");

        retryDelay = 1000;

    };

    socket.onmessage = (event) => {

        console.log(event.data);

    };

    socket.onclose = () => {

        const delay =
            retryDelay + Math.random() * 1000;

        setTimeout(connect, delay);

        retryDelay = Math.min(
            retryDelay * 2,
            MAX_DELAY
        );
```
```

```
    };

}

connect();
```

Production application-এ intentional close, authentication failure এবং temporary network failure আলাদাভাবে handle করা উচিত।

---

21. WebSocket Performance Optimization

অনেক concurrent connection থাকলে performance গুরুত্বপূর্ণ হয়ে ওঠে।

1. Use Async Programming

I/O-heavy application-এ async programming ব্যবহার করো।

2. Reduce Message Size

অপ্রয়োজনীয় বড় JSON message পাঠিও না।

3. Avoid Unnecessary Broadcasting

শুধু প্রয়োজনীয় user বা group-কে message পাঠাও।

4. Use Redis

Multiple server-এর মধ্যে message distribute করতে Redis channel layer ব্যবহার করতে পারো।

5. Limit Connections

প্রতি user বা IP-এর connection সংখ্যা সীমিত রাখো।

6. Handle Disconnects

Disconnected client-এর resources release করো।

7. Use Pagination

পুরোনো chat history একবারে না পাঠিয়ে pagination ব্যবহার করো।

8. Monitor Connections

Active connection, message throughput এবং error rate monitor করো।

---

22. WebSocket Scaling

ধরো, তোমার application-এ ১০,০০০ concurrent user রয়েছে।

একটি server দিয়ে সব connection পরিচালনা করা কঠিন হতে পারে।

তখন multiple server ব্যবহার করা যায়।

Architecture

```text
          USERS
            |
       LOAD BALANCER
            |
     ------------------
     |      |       |
  Server 1 Server 2 Server 3
     |      |       |
     ------------------
            |
          REDIS
            |
         DATABASE
```

Important Components

Load Balancer

Incoming connection বিভিন্ন server-এ distribute করে।

Multiple Application Servers

একাধিক server connection handle করে।

Redis

Server-এর মধ্যে event বা message exchange করতে সাহায্য করে।

Database

Permanent information সংরক্ষণ করে।

Sticky Sessions

কিছু deployment setup-এ একই client-এর connection নির্দিষ্ট server-এর সঙ্গে ধরে রাখার প্রয়োজন হতে পারে।

তবে WebSocket handshake সম্পন্ন হওয়ার পর connection সাধারণত সেই server-এর সঙ্গেই থাকে।

---

23. WebSocket Use Cases

1. Chat Application

Real-time messaging।

2. Live Notification

নতুন notification সঙ্গে সঙ্গে পাঠানো।

3. Online Gaming

Real-time player movement এবং game state update।

4. Live Stock Market

Stock price update।

5. Live Tracking

Delivery বা vehicle tracking।

6. Collaborative Editor

একাধিক user একই document edit করতে পারে।

7. Real-time Dashboard

Live analytics এবং monitoring।

8. Customer Support

Live customer-agent communication।

---

24. WebSocket Best Practices

1. Production-এ WSS ব্যবহার করো।
2. Authentication implement করো।
3. Authorization check করো।
4. Origin validate করো।
5. Incoming message validate করো।
6. Message size limit রাখো।
7. Rate limiting implement করো।
8. Proper error handling রাখো।
9. Reconnection strategy তৈরি করো।
10. Dead connection detect করো।
11. Unnecessary broadcasting এড়িয়ে চলো।
12. Sensitive data log করো না।
13. Database operation carefully handle করো।
14. Connection cleanup নিশ্চিত করো।
15. Monitoring এবং logging implement করো।

---

25. Common WebSocket Errors

1. Connection Refused

কারণ:
- Server চালু নেই।
- ভুল port ব্যবহার করা হয়েছে।
- Endpoint ভুল।

সমাধান:
- Server status check করো।
- URL এবং port check করো।

2. WebSocket is Closed Before the Connection is Established

কারণ:
- Server handshake reject করেছে।

- Server crash করেছে।
- Authentication failure হয়েছে।

সমাধান:
- Server logs check করো।
- Authentication এবং routing check করো।

3. 403 Forbidden

কারণ:
- Origin validation failure।
- Authentication বা permission failure।

সমাধান:
- Allowed hosts check করো।
- User authentication এবং authorization check করো।

4. 404 Not Found

কারণ:
- WebSocket routing path ভুল।

সমাধান:
- URL pattern check করো।

5. 500 Internal Server Error

কারণ:
- Server-side exception।

সমাধান:
- Application logs check করো।
- Consumer বা endpoint code debug করো।

6. WebSocketDisconnect

কারণ:
- Client connection বন্ধ করেছে।
- Network connection lost হয়েছে।

সমাধান:
- Disconnect event handle করো।

---

26. WebSocket Interview Questions

Beginner Level

Q1. What is WebSocket?

Answer:

WebSocket is a communication protocol that provides full-duplex communication between a client and server over a persistent connection.

বাংলায়:

Client এবং Server-এর মধ্যে real-time bidirectional communication তৈরি করে।

Q2. What is the difference between HTTP and WebSocket?

Answer:

HTTP follows a request-response model, while WebSocket provides full-duplex communication over a persistent connection.

Q3. What is ws://?

Answer:

It is the WebSocket URI scheme for an unencrypted WebSocket connection.

Q4. What is wss://?

Answer:

It is the secure WebSocket URI scheme that uses TLS encryption.

Q5. What is a WebSocket handshake?

Answer:

It is the initial process used to establish a WebSocket connection.

Q6. What does HTTP 101 mean?

Answer:

101 Switching Protocols indicates that the server accepted the protocol upgrade request.

Q7. What is full-duplex communication?

Answer:

Both client and server can send and receive data independently at the same time.

Intermediate Level

Q8. What is Broadcasting?

Answer:

Broadcasting means sending one message to multiple connected clients.

Q9. What is a WebSocket Group?

Answer:

A group is a logical collection of WebSocket connections that can receive shared events.

Q10. What is Redis Channel Layer?

Answer:

It is a Channels backend that uses Redis to distribute messages and events across application instances.

Q11. What is Django Channels?

Answer:

Django Channels extends Django to support asynchronous protocols such as WebSocket.

Q12. What is a Consumer?

Answer:

A Consumer is a Channels component that handles connection events and incoming messages.

Q13. What is the difference between SyncConsumer and AsyncConsumer?

Answer:

SyncConsumer uses synchronous methods, while AsyncConsumer uses asynchronous methods and supports async I/O.

Q14. What is Ping and Pong?

Answer:

Ping and Pong are WebSocket control frames used to check connection health.

Advanced Level

Q15. How do you scale WebSocket applications?

Answer:

Use multiple application servers, a load balancer, a shared message layer such as Redis, and proper connection management.

Q16. How do you secure WebSocket?

Answer:

Use WSS, authentication, authorization, origin validation, input validation, rate limiting, and message size limits.

Q17. What is Exponential Backoff?

Answer:

It is a reconnection strategy where the waiting time increases after each failed attempt.

Q18. What is the difference between SSE and WebSocket?

Answer:

SSE provides one-way server-to-client event streaming, while WebSocket provides bidirectional communication.

Q19. Does WebSocket use TCP?

Answer:

The traditional WebSocket protocol uses TCP as its transport.

Q20. Can WebSocket send binary data?

Answer:

Yes. WebSocket supports both text and binary messages.

Q21. Is WebSocket stateless?

Answer:

The protocol connection is stateful because it remains open and maintains connection state.

Q22. Does WebSocket replace REST API?

Answer:

No. REST is useful for request-response operations, while WebSocket is useful for continuous real-time communication. They can work together.

---

27. WebSocket Project Ideas

Beginner Projects

1. Echo Server

Features:
- Client message পাঠাবে।
- Server একই message ফিরিয়ে দেবে।

2. Simple Chat Application

Features:
- Send message
- Receive message
- Real-time communication

Intermediate Projects

3. Group Chat Application

Features:
- Multiple rooms
- User join/leave
- Group messaging
- Message history

4. Live Notification System

Features:
- Real-time notification
- User-specific notification
- Unread notification count

5. Live Order Tracking

Features:
- Order status update
- Delivery status
- Real-time location update

Advanced Projects

6. Real-time Collaboration Tool

Features:
- Multiple users
- Live editing
- User presence
- Shared document state

7. Live Trading Dashboard

Features:
- Live market updates
- Real-time charts
- Price notifications

8. Scalable Chat System

Features:

- Django Channels
- Redis
- PostgreSQL
- Authentication
- Multiple chat rooms
- Private messaging
- Message history
- Read receipts
- Online/offline status

---

28. WebSocket Learning Roadmap

Level 1: Fundamentals

- HTTP basics
- Client-Server architecture
- Request-Response
- WebSocket protocol
- Handshake
- ws:// and wss://

Level 2: JavaScript WebSocket

- WebSocket constructor
- onopen
- onmessage
- onerror
- onclose
- send()
- close()

Level 3: Python WebSocket

- websockets package
- Async programming
- Client
- Server
- Send and receive

Level 4: FastAPI WebSocket

- WebSocket endpoint

- accept()
- receive_text()
- send_text()
- WebSocketDisconnect
- Connection manager

Level 5: Django Channels

- ASGI
- Consumer
- Routing
- Authentication
- Groups
- Broadcasting

Level 6: Redis

- Redis fundamentals
- Channel layer
- Group messaging
- Multiple server communication

Level 7: Advanced Concepts

- Security
- Reconnection
- Heartbeat
- Rate limiting
- Scaling
- Load balancing
- Monitoring

Level 8: Build Projects

- Chat application
- Notification system
- Live dashboard
- Scalable messaging system

---

29. Important Commands

Install FastAPI

```bash
pip install fastapi uvicorn
```

Run FastAPI

```bash
uvicorn main:app --reload
```

Install Django Channels

```bash
pip install channels
```

Install Redis Channel Layer

```bash
pip install channels-redis
```

Install Daphne

```bash
pip install daphne
```

Run Django ASGI Application

```bash
daphne -p 8000 config.asgi:application
```

Install Python WebSockets

```bash
pip install websockets
```

---

30. Final Summary

Topic	Key Point
WebSocket	Real-time communication protocol
Full-Duplex	Both sides communicate
Persistent Connection	Connection remains open
Handshake	Initial connection process
HTTP 101	Protocol switching accepted
ws://	Unencrypted WebSocket
wss://	Secure WebSocket
Ping/Pong	Connection health checking
Broadcasting	Send to multiple clients
Group	Collection of connections
FastAPI	Python WebSocket support
Django Channels	Django real-time support
Consumer	Handles WebSocket events
Redis	Cross-server message distribution
ASGI	Asynchronous Server Gateway Interface
SSE	One-way server-to-client events
Scaling	Multiple servers and shared messaging

Most Important Things to Remember

1. WebSocket provides bidirectional communication.
2. WebSocket connection is persistent.
3. WebSocket starts with a handshake.
4. HTTP 101 means protocol switching accepted.
5. `ws://` is unencrypted.
6. `wss://` is encrypted.
7. FastAPI supports WebSocket directly.
8. Django uses Channels for WebSocket support.
9. Redis helps distribute messages between servers.
10. Authentication and authorization are essential.
11. Production applications should use WSS.
12. WebSocket does not replace REST API.

Recommended Documentation

1. MDN WebSocket API:
   https://developer.mozilla.org/en-US/docs/Web/API/WebSocket

2. FastAPI WebSocket:

https://fastapi.tiangolo.com/advanced/websockets/

3. Django Channels:
https://channels.readthedocs.io/

4. Python WebSockets:
https://websockets.readthedocs.io/

5. WebSocket Protocol (RFC 6455):
https://www.rfc-editor.org/rfc/rfc6455

6. Redis:
https://redis.io/docs/latest/

End of WebSocket Complete Notes

Practice Goal: Build a real-time Chat Application using Django Channels + Redis + PostgreSQL.